



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 6ο: Βάσεις Δεδομένων

Σταύρος Μέμος – 03121837
Αθανάσιος Μητρόπουλος – 03119959

Εξαμηνιαία Εργασία: "Υγειοpolis General Hospital"

Περίληψη

Η παρούσα εργαστηριακή αναφορά αφορά την εξαμηνιαία εργασία στα πλαίσια του εργαστηρίου του μαθήματος Βάσεις Δεδομένων. Σχεδιάσαμε και υλοποιήσαμε ένα ολοκληρωμένο σύστημα αποθήκευσης και διαχείρισης πληροφοριών για τη λειτουργία του Γενικού Νοσοκομείου «Υγειοpolis». Το σύστημα καλύπτει τη διαχείριση προσωπικού, ασθενών, νοσηλειών, εφημεριών, ιατρικών πράξεων, συνταγογραφήσεων και αξιολογήσεων, ενώ συνοδεύεται από πλήρη διαδικτυακή εφαρμογή.

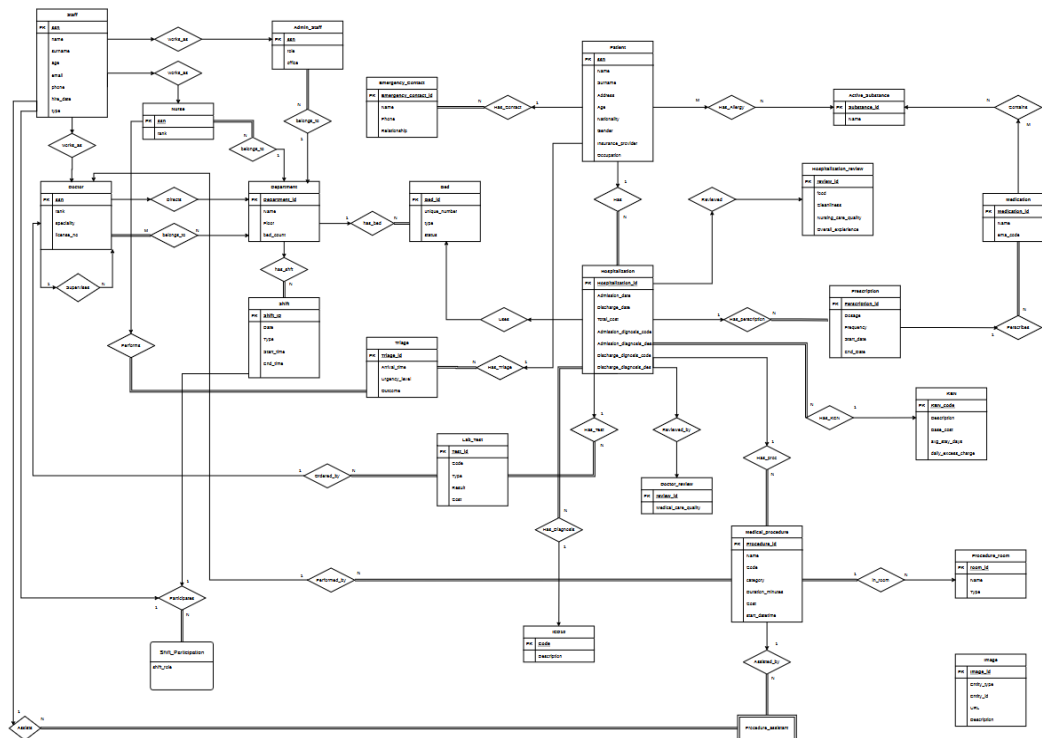
Περιεχόμενα

A. Σχεδιασμός και υλοποίηση Βάσης Δεδομένων	2
A.1. Διάγραμμα Οντοτήτων – Συσχετίσεων	2
A.2. Σχεσιακό Διάγραμμα	4
A.2.1. Περιορισμοί και Triggers	5
A.2.2. Ευρετήρια (Indexes)	5
A.3. Fake Data	5
B. Queries	6
Γ. Υλοποίηση Εφαρμογής	17

A. Σχεδιασμός και υλοποίηση Βάσης Δεδομένων

A.1. Διάγραμμα Οντοτήτων – Συσχετίσεων (ER Diagram)

Σχεδιάσαμε το ER διάγραμμα βάσει της εκφώνησης. Χρησιμοποιούνται ορθογώνια για οντότητες και attributes, ρόμβοι για relationships, μονές γραμμές για μερική συμμετοχή και διπλές για ολική. Το διάγραμμα περιλαμβάνει 25 οντότητες με όλες τις μεταξύ τους σχέσεις.



Εικόνα 1: ER Diagram – Ygeiopolis General Hospital

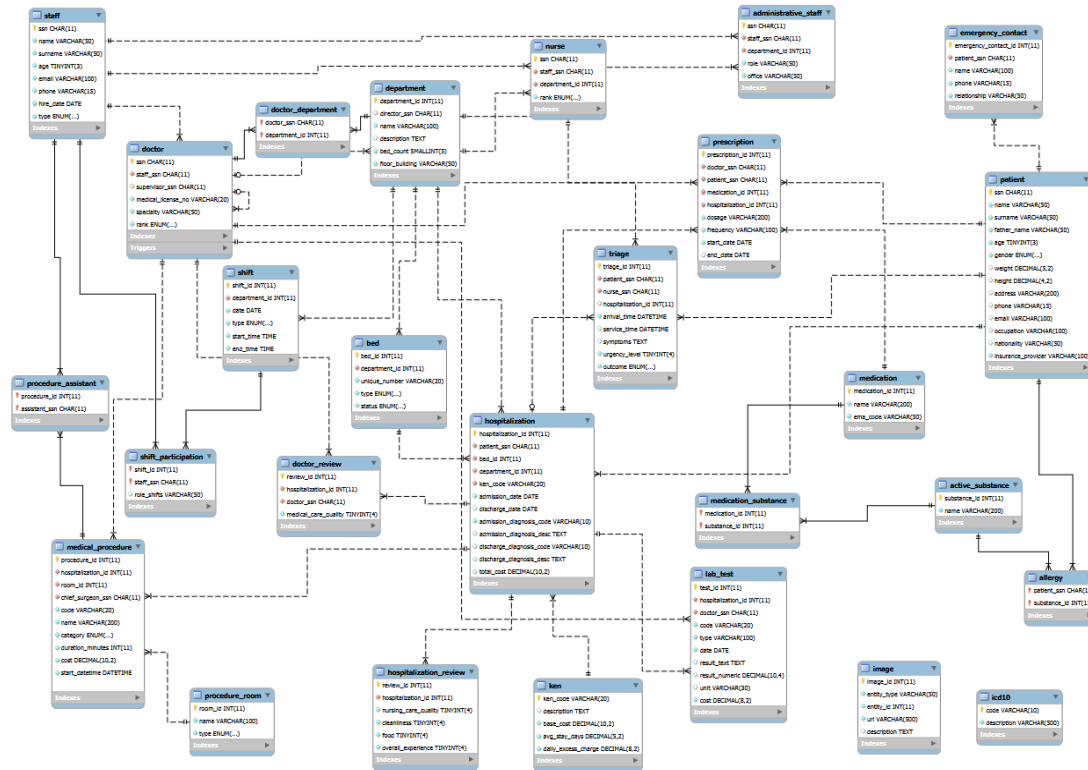
Παρακάτω παρουσιάζονται οι σχέσεις μεταξύ των επιμέρους οντοτήτων:

- Staff – Doctor / Nurse / Admin_Staff (works_as, 1:1): Το προσωπικό κατηγοριοποιείται σε τρεις υποκατηγορίες με ολική συμμετοχή και από τις δύο πλευρές.
- Doctor – Department (belongs_to, M:N): Κάθε ιατρός μπορεί να ανήκει σε πολλά τμήματα. Υλοποιείται μέσω doctor_department.
- Department – Doctor (Directs, 1:1): Κάθε τμήμα διευθύνεται από έναν DIRECTOR.
- Nurse / Admin_Staff – Department (belongs_to, N:1): Κάθε νοσηλεύτης/διοικητικός ανήκει σε ένα τμήμα.
- Doctor – Doctor (Supervises, 1:N): Αναδρομική σχέση εποπτείας. Οι DIRECTOR δεν έχουν επόπτη.
- Department – Shift (has_shift, 1:N): Κάθε εφημερία ανήκει σε ένα τμήμα.
- Staff – Shift_Participation – Shift (Participates, M:N): Ενδιάμεσος πίνακας με attribute role_shifts.
- Department – Bed (has_bed, 1:N): Κάθε κλίνη ανήκει σε ένα τμήμα.
- Patient – Hospitalization (Has, 1:N): Κάθε ασθενής μπορεί να έχει πολλές νοσηλείες.
- Patient – Triage (Has_Triage, 1:N): Κάθε ασθενής μπορεί να έχει πολλά περιστατικά triage.
- Hospitalization – KEN (Has_KEN, N:1): Κάθε νοσηλεία αντιστοιχεί σε έναν KEN κωδικό.
- Hospitalization – ICD10 (Has_Diagnosis, N:1): Κάθε νοσηλεία συνδέεται με κωδικό ICD-10.

- Hospitalization – Doctor_Review (Reviewed_by, 1:N): Αξιολόγηση ιατρού ανά νοσηλεία.
- Hospitalization – Hospitalization_Review (Reviewed, 1:1): Μία αξιολόγηση νοσηλείας μετά την έξοδο.
- Patient – Active_Substance (Has_Allergy, M:N): Αλλεργίες ασθενών σε δραστικές ουσίες.
- Medication – Active_Substance (Contains, M:N): Σύνθεση φαρμάκων από δραστικές ουσίες.
- Image (Polymorphic): Χρησιμοποιεί entity_type και entity_id για polymorphic relationship χωρίς FK constraint.

A.2. Σχεσιακό Διάγραμμα και υλοποίηση Βάσης Δεδομένων (Relational Model)

Το σχεσιακό διάγραμμα δημιουργήθηκε με τη μετατροπή του ER διαγράμματος. Οι 1:N σχέσεις υλοποιούνται με FKs, οι M:N με ενδιάμεσους πίνακες (doctor_department, medication_substance, procedure_assistant, shift_participation, allergy).



Εικόνα 2: Σχεσιακό Διάγραμμα – Ygeiopolis General Hospital

A.2.1. Απαραίτητοι Περιορισμοί για ορθότητα

Constraints

Ορίσαμε τους παρακάτω περιορισμούς ακεραιότητας:

- CHECK (type IN ('DOCTOR','NURSE','ADMINISTRATIVE')): Τύπος μέλους προσωπικού.
- CHECK (rank IN ('RESIDENT','JUNIOR_CONSULTANT','SENIOR_CONSULTANT','DIRECTOR')): Βαθμίδες ιατρού.
- CHECK (rank IN ('NURSE_ASSISTANT','NURSE','HEAD_NURSE')): Βαθμίδες νοσηλεύτη.
- CHECK (gender IN ('MALE','FEMALE','OTHER')): Φύλο ασθενή.
- CHECK (urgency_level BETWEEN 1 AND 5): Επίπεδο επείγοντος triage.
- CHECK (medical_care_quality BETWEEN 1 AND 5): Αξιολόγηση ιατρικής φροντίδας.
- CHECK (total_cost >= 0): Κόστος νοσηλείας.
- CHECK (duration_minutes > 0): Διάρκεια ιατρικής πράξης.
- CHECK (status IN ('AVAILABLE','OCCUPIED','MAINTENANCE')): Κατάσταση κλίνης.
- CHECK (outcome IN ('DISCHARGED','ADMITTED')): Αποτέλεσμα triage.
- UNIQUE (department_id, date, type): Μοναδική βάρδια ανά τμήμα/ημέρα/τύπο.

Triggers

Υλοποιήσαμε 7 triggers για αυτόματη επιβολή των επιχειρησιακών κανόνων:

- `trg_no_circular_supervision` (BEFORE INSERT ON `doctor`): Αποτρέπει κυκλικές αλυσίδες εποπτείας ακολουθώντας αναδρομικά την αλυσίδα μέχρι βάθος 100.
- `trg_resident_must_have_supervisor` (BEFORE INSERT ON `doctor`): Κάθε RESIDENT πρέπει να έχει `supervisor_ssn`.
- `trg_director_no_supervisor` (BEFORE INSERT ON `doctor`): Κανένας DIRECTOR δεν μπορεί να έχει επόπτη.
- `trg_no_allergy_prescription` (BEFORE INSERT ON `prescription`): Αποτρέπει συνταγογράφηση αν ο ασθενής έχει αλλεργία σε δραστική ουσία του φαρμάκου.
- `trg_max_shifts_per_month` (BEFORE INSERT ON `shift_participation`): Έλεγχος μέγιστων βαρδιών ανά μήνα — Ιατροί 15, Νοσηλευτές 20, Διοικητικοί 25.
- `trg_min_rest_between_shifts` (BEFORE INSERT ON `shift_participation`): Ελάχιστο 8ωρο ανάπαυσης μεταξύ βαρδιών.
- `trg_max_consecutive_night_shifts` (BEFORE INSERT ON `shift_participation`): Μέγιστο 3 συνεχόμενες νυχτερινές βάρδιες.

Χρησιμοποιούμε `DELIMITER //` για να αλλάξουμε το τερματιστικό σύμβολο κατά τη δημιουργία triggers, αποφεύγοντας σύγκρουση με τα `;` εντός του κώδικα του trigger.

A.2.2. Ορισμός Ευρετηρίων (Indexes)

Εκτός από τα αυτόματα indexes σε PKs και FKs (InnoDB), ορίσαμε στοχευμένα indexes:

- `INDEX fk_nos_patient` ON `hospitalization(patient_ssn)`: Αναζήτηση νοσηλειών ανά ασθενή — Q06.
- `INDEX fk_axi_doctor` ON `doctor_review(doctor_ssn)`: Αναζήτηση αξιολογήσεων ανά ιατρό — Q04.
- `INDEX uq_axiolog_iat` ON `doctor_review(hospitalization_id, doctor_ssn)`: Composite index — Q04.
- `INDEX fk_praxi_xeirourgios` ON `medical_procedure(chief_surgeon_ssn)`: Πράξεις ανά χειρουργό — Q02, Q05, Q11.
- `INDEX fk_syn_patient` ON `prescription(patient_ssn)`: Συνταγές ανά ασθενή — Q07, Q10.

A.3. Fake Data

Τα δεδομένα δοκιμής δημιουργήθηκαν με Python script (βιβλιοθήκες `faker` και `random`) και αποθηκεύτηκαν στο `load.sql`. Δημιουργήθηκαν: 80 ιατροί, 100 νοσηλευτές, 50 διοικητικοί, 15 τμήματα, 324 κλίνες, 100 ασθενείς, νοσηλείες με ICD-10 και KEN κωδικούς, 282 φάρμακα από τη βάση EMA Article 57, εφημερίες, ιατρικές πράξεις, εργαστηριακές εξετάσεις, συνταγογραφήσεις, αξιολογήσεις και triage περιστατικά. Τα δεδομένα ικανοποιούν όλους τους περιορισμούς που υπαγορεύονται από την εκφώνηση και τις σχεδιαστικές παραδοχές.

B. Queries

B.1 Βρείτε τα έσοδα ανά τμήμα, έτος, KEN και ασφαλιστικό φορέα.

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```
1  SELECT
2      d.name AS department,
3      YEAR(h.admission_date) AS year,
4      h.ken_code,
5      k.base_cost,
6      COUNT(h.hospitalization_id) AS total_hospitalizations,
7      SUM(k.base_cost) AS total_base_cost,
8      SUM(h.total_cost - k.base_cost) AS extra_charge,
9      SUM(h.total_cost) AS total_revenue,
10     p.insurance_provider,
11     COUNT(p.ssn) AS patients_per_provider
12 FROM hospitalization h
13 JOIN department d ON h.department_id = d.department_id
14 JOIN ken k ON h.ken_code = k.ken_code
15 JOIN patient p ON h.patient_ssn = p.ssn
16 GROUP BY d.department_id, YEAR(h.admission_date), h.ken_code, p.insurance_provider
17 ORDER BY year, department, h.ken_code;
```

Αρχικά κάνουμε JOIN από hospitalization → department, ken, patient για να συνδυάσουμε τα έσοδα με το τμήμα, τον KEN κωδικό και τον ασφαλιστικό φορέα. Ομαδοποιούμε ανά department_id, YEAR(admission_date), ken_code και insurance_provider. Χρησιμοποιούμε SUM για τα βασικά κόστη και τα έξτρα έσοδα, COUNT για τον αριθμό νοσηλείων και COUNT(DISTINCT p.ssn) για τους ασθενείς ανά ασφαλιστικό φορέα. Το ORDER BY ταξινομεί ανά έτος, τμήμα και KEN.

B.2 Βρείτε ιατρούς με ειδικότητα, εφημερίες και ιατρικές πράξεις για το τρέχον έτος.

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```
1  SELECT
2      doc.ssn,
3      s.name,
4      s.surname,
5      doc.specialty,
6      doc.rank,
7      CASE WHEN COUNT(DISTINCT sp.shift_id) > 0 THEN 'YES' ELSE 'NO' END AS had_shift,
8      COUNT(DISTINCT mp.procedure_id) AS total_procedures
9 FROM doctor doc
10 JOIN staff s ON doc.ssn = s.ssn
11 LEFT JOIN shift_participation sp ON doc.ssn = sp.staff_ssn
12     AND sp.shift_id IN (
13         SELECT shift_id FROM shift WHERE YEAR(date) = YEAR(CURDATE())
14     )
15 LEFT JOIN medical_procedure mp ON doc.ssn = mp.chief_surgeon_ssn
16 WHERE doc.specialty = 'Καρδιολογία'
17 GROUP BY doc.ssn, s.name, s.surname, doc.specialty, doc.rank
18 ORDER BY total_procedures DESC;
```

Ξεκινάμε από τον πίνακα doctor και κάνουμε JOIN με staff για το ονοματεπώνυμο. Χρησιμοποιούμε LEFT JOIN με shift_participation ώστε να συμπεριλάβουμε ιατρούς χωρίς εφημερίες, φιλτράροντας με subquery τις εφημερίες του τρέχοντος έτους (YEAR(date) = YEAR(CURDATE())). Αντίστοιχα LEFT JOIN με medical_procedure για τον αριθμό πράξεων. Το CASE WHEN COUNT(DISTINCT sp.shift_id) > 0 THEN 'YES' ELSE 'NO' END εμφανίζει αν είχε εφημερία. Φιλτράρουμε βάσει ειδικότητας ('Καρδιολογία' στο παράδειγμα).

B.3 Βρείτε ασθενείς που νοσηλεύτηκαν περισσότερες από 3 φορές στο ίδιο τμήμα.

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```
1  SELECT
2      p.ssn,
3      p.name,
4      p.surname,
5      d.name AS department,
6      COUNT(h.hospitalization_id) AS total_hospitalizations,
7      SUM(h.total_cost) AS total_cost
8  FROM patient p
9  JOIN hospitalization h ON p.ssn = h.patient_ssn
10 JOIN department d ON h.department_id = d.department_id
11 GROUP BY p.ssn, p.name, p.surname, h.department_id, d.name
12 HAVING COUNT(h.hospitalization_id) > 3
13 ORDER BY total_hospitalizations DESC;
```

Κάνουμε JOIN patient → hospitalization → department και ομαδοποιούμε ανά ασθενή και τμήμα (GROUP BY p.ssn, p.name, p.surname, h.department_id, d.name). Χρησιμοποιούμε HAVING COUNT(h.hospitalization_id) > 3 για να κρατήσουμε μόνο τους ασθενείς που νοσηλεύτηκαν περισσότερες από 3 φορές στο ίδιο τμήμα. Το SUM(h.total_cost) δείχνει το συνολικό κόστος ανά ασθενή/τμήμα.

B.4 Για κάποιο ιατρό, βρείτε τον μέσο όρο αξιολογήσεων ιατρικής φροντίδας και τον μέσο όρο των κριτηρίων αξιολόγησης νοσηλείας.

Υλοποιούμε το παραπάνω ερώτημα για τον ιατρό με ssn='0000000007':

```
1  SELECT
2      doc.ssn,
3      s.name,
4      s.surname,
5      ROUND(AVG(dr.medical_care_quality), 2) AS avg_doctor_rating,
6      ROUND(AVG(hr.nursing_care_quality), 2) AS avg_nursing_care,
7      ROUND(AVG(hr.cleanliness), 2) AS avg_cleanliness,
8      ROUND(AVG(hr.food), 2) AS avg_food,
9      ROUND(AVG(hr.overall_experience), 2) AS avg_overall_experience
10 FROM doctor doc
11 JOIN staff s ON doc.ssn = s.ssn
12 JOIN doctor_review dr ON doc.ssn = dr.doctor_ssn
13 JOIN hospitalization h ON dr.hospitalization_id = h.hospitalization_id
14 LEFT JOIN hospitalization_review hr ON h.hospitalization_id = hr.hospitalization_id
15 WHERE doc.ssn = '0000000007'
16 GROUP BY doc.ssn, s.name, s.surname;
```

Ξεκινάμε από τον doctor, κάνουμε JOIN με staff για το ονοματεπώνυμο, JOIN με doctor_review για τις αξιολογήσεις, JOIN με hospitalization για σύνδεση με νοσηλείες και LEFT JOIN με hospitalization_review για τα κριτήρια νοσηλείας. Χρησιμοποιούμε ROUND(AVG(...), 2) για τον μέσο όρο με 2 δεκαδικά.

Στο ερώτημα αυτό παρουσιάζουμε εναλλακτικό Query Plan με χρήση EXPLAIN και FORCE INDEX. Το EXPLAIN μας ενημερώνει για: id (αύξων αριθμός), table (πίνακας), type (const/eq_ref/ref/ALL), key (index που επελέγη), rows (εκτιμώμενες εγγραφές), Extra (πρόσθετες πληροφορίες).

(α) EXPLAIN – Κανονικό query:

id	table	type	key	rows	Extra
1	doc	const	PRIMARY	1	Using index
1	s	const	PRIMARY	1	
1	dr	ref	fk_axi_doctor	3	Using index condition
1	h	eq_ref	PRIMARY	1	Using index
1	hr	eq_ref	hospitalization_id	1	

(β) EXPLAIN – Με FORCE INDEX (uq_axiolog_iat):

id	table	type	key	rows	Extra
1	doc	const	PRIMARY	1	Using index
1	s	const	PRIMARY	1	
1	dr	ALL	NULL	147	Using where
1	h	eq_ref	PRIMARY	1	Using index
1	hr	eq_ref	hospitalization_id	1	

(γ) Σύγκριση:

Εκδοχή	Rows	Type	Αποτέλεσμα
Κανονική (fk_axi_doctor)	3	ref	Βέλτιστη — index scan
FORCE INDEX (uq_axiolog_iat)	147	ALL (full scan)	Υποβαθμισμένη (~49x πιο αργή)

(δ) Ανάλυση:

Ο βελτιστοποιητής επέλεξε αυτόματα το fk_axi_doctor σκανάροντας 3 εγγραφές. Με FORCE INDEX στο uq_axiolog_iat, πραγματοποιείται full table scan 147 εγγραφών (~49x υποβάθμιση), γιατί το composite index έχει hospitalization_id ως πρώτο πεδίο και δεν εξυπηρετεί φιλτράρισμα μόνο βάσει doctor_ssn.

B.5 Βρείτε τους νέους ιατρούς (ηλικία < 35) με τις περισσότερες χειρουργικές πράξεις.

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```
1  SELECT
2      doc.ssn,
3      s.name,
4      s.surname,
5      s.age,
6      doc.specialty,
7      doc.rank,
8      COUNT(mp.procedure_id) AS total_surgical
9  FROM doctor doc
10 JOIN staff s ON doc.ssn = s.ssn
11 JOIN medical_procedure mp ON doc.ssn = mp.chief_surgeon_ssn
12 WHERE s.age < 35
13 AND mp.category = 'SURGICAL'
14 GROUP BY doc.ssn, s.name, s.surname, s.age, doc.specialty, doc.rank
15 ORDER BY total_surgical DESC;
```

Κάνουμε JOIN doctor → staff → medical_procedure φιλτράροντας με WHERE s.age < 35 AND mp.category = 'SURGICAL'. Ομαδοποιούμε ανά ιατρό και χρησιμοποιούμε COUNT(mp.procedure_id) AS total_surgical για τον αριθμό χειρουργικών πράξεων. Το ORDER BY total_surgical DESC φέρνει πρώτους τους ιατρούς με τις περισσότερες πράξεις.

B.6 Για κάποιο ασθενή, βρείτε το ιστορικό νοσηλείων του με τις διαγνώσεις ICD-10, το κόστος και τον μέσο όρο αξιολόγησης.

Υλοποιούμε το παραπάνω ερώτημα για τον ασθενή με ssn='00000000657':

```
1  SELECT
2      p.ssn,
3      p.name,
4      p.surname,
5      h.hospitalization_id,
6      h.admission_date,
7      h.discharge_date,
8      h.admission_diagnosis_code,
9      icd.description AS diagnosis,
10     h.total_cost,
11     ROUND(AVG(hr.overall_experience), 2) AS avg_overall_experience
12 FROM patient p
13 JOIN hospitalization h ON h.patient_ssn = p.ssn
14 JOIN icd10 icd ON h.admission_diagnosis_code = icd.code
15 LEFT JOIN hospitalization_review hr ON h.hospitalization_id = hr.hospitalization_id
16 WHERE p.ssn = '00000000657'
17 GROUP BY p.ssn, p.name, p.surname, h.hospitalization_id, h.admission_date,
18          h.discharge_date, h.admission_diagnosis_code, icd.description, h.total_cost
19 ORDER BY h.admission_date;
```

Ξεκινάμε από patient, κάνουμε JOIN με hospitalization για τις νοσηλείες, JOIN με icd10 για την περιγραφή διάγνωσης, και LEFT JOIN με hospitalization_review για τον μέσο όρο αξιολόγησης. Ο

GROUP BY εξασφαλίζει μία γραμμή ανά νοσηλεία και το ORDER BY admission_date ταξινομεί χρονολογικά.

(α) EXPLAIN – Κανονικό query:

id	table	type	key	rows	Extra
1	p	const	PRIMARY	1	Using temporary; Using filesort
1	h	ref	fk_nos_patient	6	Using index condition; Using where
1	hr	eq_ref	hospitalization_id	1	
1	icd	eq_ref	PRIMARY	1	

(β) EXPLAIN – Με FORCE INDEX (PRIMARY) στο hospitalization:

id	table	type	key	rows	Extra
1	p	const	PRIMARY	1	Using temporary; Using filesort
1	h	ALL	NULL	503	Using where
1	hr	eq_ref	hospitalization_id	1	
1	icd	eq_ref	PRIMARY	1	

(γ) Σύγκριση:

Εκδοση	Rows	Type	Αποτέλεσμα
Κανονική (fk_nos_patient)	6	ref	Βέλτιστη — index scan
FORCE INDEX (PRIMARY)	503	ALL (full scan)	Υποβαθμισμένη (~84x πιο αργή)

(δ) Ανάλυση:

Ο βελτιστοποιητής επέλεξε αυτόματα το fk_nos_patient σκανάροντας 6 εγγραφές. Με FORCE INDEX στο PRIMARY, πραγματοποιείται full table scan 503 εγγραφών (~84x υποβάθμιση), γιατί το PRIMARY key είναι το hospitalization_id και δεν εξυπηρετεί φιλτράρισμα βάσει patient_ssn.

B.7 Βρείτε τις δραστικές ουσίες με τον αριθμό αλλεργικών ασθενών και φαρμάκων που τις περιέχουν.

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```
1  SELECT
2      a.substance_id,
3      a.name AS active_substance,
4      COUNT(DISTINCT al.patient_ssn) AS allergic_patients,
5      COUNT(DISTINCT ms.medication_id) AS medications_count
6  FROM active_substance a
7  LEFT JOIN allergy al ON a.substance_id = al.substance_id
8  LEFT JOIN medication_substance ms ON a.substance_id = ms.substance_id
9  GROUP BY a.substance_id, a.name
10 ORDER BY allergic_patients DESC;
```

Ξεκινάμε από τον πίνακα active_substance και κάνουμε LEFT JOIN με allergy για τους αλλεργικούς ασθενείς και LEFT JOIN με medication_substance για τα φάρμακα. Χρησιμοποιούμε COUNT(DISTINCT al.patient_ssn) για τον αριθμό μοναδικών αλλεργικών ασθενών και COUNT(DISTINCT ms.medication_id) για τον αριθμό φαρμάκων. Ομαδοποιούμε ανά δραστική ουσία και ταξινομούμε κατά φθίνουσα σειρά αλλεργικών ασθενών.

B.8 Βρείτε το προσωπικό που δεν έχει εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

Υλοποιούμε το παραπάνω ερώτημα για ημερομηνία '2024-05-29' και τμήμα 1:

```
1  SELECT
2      s.ssn,
3      s.name,
4      s.surname,
5      s.type
6  FROM staff s
7  WHERE s.ssn NOT IN (
8      SELECT sp.staff_ssn
9      FROM shift_participation sp
10     JOIN shift sh ON sp.shift_id = sh.shift_id
11     WHERE sh.date = '2024-05-29'
12     AND sh.department_id = 1
13 )
14 ORDER BY s.type, s.surname;
```

Επιλέγουμε από τον πίνακα staff όλα τα μέλη που δεν εμφανίζονται στον πίνακα shift_participation για τη συγκεκριμένη ημερομηνία και τμήμα. Χρησιμοποιούμε NOT IN με subquery που βρίσκει τα staff_ssn που συμμετέχουν σε εφημερία (JOIN shift) του συγκεκριμένου τμήματος και ημερομηνίας. Το ORDER BY s.type, s.surname ταξινομεί ανά τύπο και επώνυμο.

B.9 Βρείτε ζεύγη ασθενών που νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε ένα έτος (>15 ημέρες).

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```

1  SELECT
2      p1.ssn AS ssn1,
3      p1.name AS name1,
4      p1.surname AS surname1,
5      p2.ssn AS ssn2,
6      p2.name AS name2,
7      p2.surname AS surname2,
8      SUM(DATEDIFF(h1.discharge_date, h1.admission_date)) AS total_days,
9      YEAR(h1.admission_date) AS year
10 FROM patient p1
11 JOIN patient p2 ON p1.ssn < p2.ssn
12 JOIN hospitalization h1 ON p1.ssn = h1.patient_ssn
13 JOIN hospitalization h2 ON p2.ssn = h2.patient_ssn
14 WHERE YEAR(h1.admission_date) = YEAR(h2.admission_date)
15 AND h1.discharge_date IS NOT NULL
16 AND h2.discharge_date IS NOT NULL
17 GROUP BY p1.ssn, p2.ssn, YEAR(h1.admission_date)
18 HAVING SUM(DATEDIFF(h1.discharge_date, h1.admission_date)) = SUM(DATEDIFF(h2.discharge_date, h2.admission_date))
19 AND SUM(DATEDIFF(h1.discharge_date, h1.admission_date)) > 15
20 ORDER BY total_days DESC;

```

Κάνουμε self JOIN στον πίνακα patient (p1, p2) με $p1.ssn < p2.ssn$ για να αποφύγουμε διπλές εμφανίσεις. Για κάθε ζεύγος κάνουμε JOIN με hospitalization (h1, h2) αντίστοιχα. Φιλτράρουμε με $WHERE YEAR(h1.admission_date) = YEAR(h2.admission_date)$ για το ίδιο έτος και με $discharge_date IS NOT NULL$ για ολοκληρωμένες νοσηλείες. Το HAVING ελέγχει ότι τα συνολικά $SUM(DATEDIFF)$ είναι ίσα και > 15 .

B.10 Βρείτε τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφούνται μαζί στην ίδια νοσηλεία.

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```

1  SELECT
2      a1.name AS substance_1,
3      a2.name AS substance_2,
4      COUNT(*) AS frequency
5  FROM prescription pr1
6  JOIN prescription pr2 ON pr1.patient_ssn = pr2.patient_ssn
7      AND pr1.hospitalization_id = pr2.hospitalization_id
8      AND pr1.medication_id < pr2.medication_id
9  JOIN medication_substance ms1 ON pr1.medication_id = ms1.medication_id
10 JOIN medication_substance ms2 ON pr2.medication_id = ms2.medication_id
11 JOIN active_substance a1 ON ms1.substance_id = a1.substance_id
12 JOIN active_substance a2 ON ms2.substance_id = a2.substance_id
13 WHERE ms1.substance_id < ms2.substance_id
14 GROUP BY a1.substance_id, a2.substance_id, a1.name, a2.name
15 ORDER BY frequency DESC
16 LIMIT 3;

```

Κάνουμε self JOIN στον πίνακα prescription (pr1, pr2) με $pr1.patient_ssn = pr2.patient_ssn$ AND $pr1.hospitalization_id = pr2.hospitalization_id$ AND $pr1.medication_id < pr2.medication_id$ για να

αποφύγουμε αντίστροφα ζεύγη. Κάνουμε JOIN με medication_substance (ms1, ms2) και active_substance (a1, a2) για να πάρουμε τα ονόματα. Το WHERE ms1.substance_id < ms2.substance_id αποφεύγει επίσης διπλούς συνδυασμούς. Ομαδοποιούμε ανά ζεύγος και κρατάμε τα 3 πρώτα με LIMIT 3.

B.11 Βρείτε ιατρούς που έχουν τουλάχιστον 5 λιγότερες πράξεις από τον μέγιστο (έτος 2024).

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```
1  SELECT
2      doc.ssn,
3      s.name,
4      s.surname,
5      doc.specialty,
6      COUNT(mp.procedure_id) AS total_procedures
7  FROM doctor doc
8  JOIN staff s ON doc.ssn = s.ssn
9  LEFT JOIN medical_procedure mp ON doc.ssn = mp.chief_surgeon_ssn
10     AND YEAR(mp.start_datetime) = 2024
11  GROUP BY doc.ssn, s.name, s.surname, doc.specialty
12  HAVING COUNT(mp.procedure_id) <= (
13      SELECT MAX(cnt) - 5
14      FROM (
15          SELECT COUNT(procedure_id) AS cnt
16          FROM medical_procedure
17          WHERE YEAR(start_datetime) = 2024
18          GROUP BY chief_surgeon_ssn
19      ) AS counts
20  )
21  ORDER BY total_procedures DESC;
```

Ξεκινάμε από doctor και κάνουμε LEFT JOIN με medical_procedure φιλτράροντας YEAR(mp.start_datetime) = 2024. Ομαδοποιούμε ανά ιατρό και χρησιμοποιούμε HAVING COUNT(mp.procedure_id) <= (subquery) για να κρατήσουμε μόνο τους ιατρούς με αριθμό πράξεων ≤ MAX - 5. Το subquery υπολογίζει το MAX(cnt) - 5 από όλους τους ιατρούς του 2024.

B.12 Βρείτε το προσωπικό ανά βάρδια/τμήμα για συγκεκριμένη εβδομάδα με ανάλυση ανά υποκατηγορία.

Υλοποιούμε το παραπάνω ερώτημα για την εβδομάδα 2024-01-01 έως 2024-01-07:

```

1  SELECT
2      d.name AS department,
3      sh.date,
4      sh.type AS shift,
5      s.type AS staff_type,
6      CASE
7          WHEN s.type = 'DOCTOR' THEN doc.specialty
8          WHEN s.type = 'NURSE' THEN n.rank
9          WHEN s.type = 'ADMINISTRATIVE' THEN ads.role
10     END AS subclass,
11     COUNT(sp.staff_ssn) AS total
12 FROM shift sh
13 JOIN department d ON sh.department_id = d.department_id
14 JOIN shift_participation sp ON sh.shift_id = sp.shift_id
15 JOIN staff s ON sp.staff_ssn = s.ssn
16 LEFT JOIN doctor doc ON s.ssn = doc.ssn
17 LEFT JOIN nurse n ON s.ssn = n.staff_ssn
18 LEFT JOIN administrative_staff ads ON s.ssn = ads.staff_ssn
19 WHERE sh.date BETWEEN '2024-01-01' AND '2024-01-07'
20 GROUP BY d.department_id, d.name, sh.date, sh.type, s.type, subclass
21 ORDER BY d.name, sh.date, sh.type, s.type;

```

Κάνουμε JOIN shift → department → shift_participation → staff και LEFT JOIN με doctor, nurse, administrative_staff για να πάρουμε την υποκατηγορία. Χρησιμοποιούμε CASE WHEN s.type = 'DOCTOR' THEN doc.specialty WHEN s.type = 'NURSE' THEN n.rank WHEN s.type = 'ADMINISTRATIVE' THEN ads.role END AS subclass για να εμφανίσουμε την ειδικότητα/βαθμίδα/ρόλο. Φιλτράρουμε με WHERE sh.date BETWEEN '2024-01-01' AND '2024-01-07'.

B.13 Βρείτε την ιεραρχία εποπτείας ενός ιατρού (αναδρομικό CTE).

Υλοποιούμε το παραπάνω ερώτημα για τον ιατρό με ssn='00000000007':

```

1  WITH RECURSIVE supervision_hierarchy AS (
2      SELECT
3          d.ssn,
4          s.name,
5          s.surname,
6          d.specialty,
7          d.rank,
8          d.supervisor_ssn,
9          0 AS level
10     FROM doctor d
11     JOIN staff s ON d.ssn = s.ssn
12     WHERE d.ssn = '00000000007'
13
14     UNION ALL
15
16     SELECT
17         d.ssn,
18         s.name,
19         s.surname,
20         d.specialty,
21         d.rank,
22         d.supervisor_ssn,
23         sh.level + 1
24     FROM doctor d
25     JOIN staff s ON d.ssn = s.ssn
26     JOIN supervision_hierarchy sh ON d.ssn = sh.supervisor_ssn
27 )
28 SELECT
29     ssn,
30     name,
31     surname,
32     specialty,
33     rank,
34     level
35 FROM supervision_hierarchy
36 ORDER BY level;

```

Το ερώτημα αυτό χρησιμοποιεί Recursive CTE (Common Table Expression) για να ακολουθήσει αναδρομικά την αλυσίδα εποπτείας από έναν συγκεκριμένο ιατρό προς τα πάνω. Το anchor member επιλέγει τον αρχικό ιατρό (level=0). Το recursive member κάνει JOIN του doctor με το CTE βρίσκοντας τον επόπτη (d.ssn = sh.supervisor_ssn) αυξάνοντας το level κατά 1 σε κάθε επανάληψη. Η αναδρομή σταματά όταν δεν βρεθεί επόπτης (NULL supervisor_ssn στους DIRECTOR). Το αποτέλεσμα δείχνει όλους τους ιατρούς της ιεραρχίας με το επίπεδό τους (0=ο ίδιος, 1=άμεσος επόπτης κλπ).

B.14 Βρείτε ICD-10 κατηγορίες με τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη (≥ 5 ανά έτος).

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```

1  SELECT
2      h1.admission_diagnosis_code,
3      i.description,
4      YEAR(h1.admission_date) AS year1,
5      COUNT(DISTINCT h1.hospitalization_id) AS count_year1,
6      YEAR(h2.admission_date) AS year2,
7      COUNT(DISTINCT h2.hospitalization_id) AS count_year2
8  FROM hospitalization h1
9  JOIN hospitalization h2
10     ON h1.admission_diagnosis_code = h2.admission_diagnosis_code
11     AND YEAR(h2.admission_date) = YEAR(h1.admission_date) + 1
12  JOIN icd10 i ON h1.admission_diagnosis_code = i.code
13  WHERE YEAR(h1.admission_date) < YEAR(h2.admission_date)
14  GROUP BY h1.admission_diagnosis_code, i.description, YEAR(h1.admission_date), YEAR(h2.admission_date)
15  HAVING COUNT(DISTINCT h1.hospitalization_id) = COUNT(DISTINCT h2.hospitalization_id)
16  AND COUNT(DISTINCT h1.hospitalization_id) >= 5
17  ORDER BY year1, count_year1 DESC;

```

Κάνουμε self JOIN στον πίνακα hospitalization (h1, h2) με βάση τον ίδιο κωδικό ICD-10 (h1.admission_diagnosis_code = h2.admission_diagnosis_code) και συνεχόμενα έτη (YEAR(h2.admission_date) = YEAR(h1.admission_date) + 1). Κάνουμε JOIN με icd10 για την περιγραφή. Ομαδοποιούμε ανά κωδικό και έτη και εφαρμόζουμε HAVING COUNT(DISTINCT h1.hospitalization_id) = COUNT(DISTINCT h2.hospitalization_id) AND COUNT(DISTINCT h1.hospitalization_id) >= 5 για να κρατήσουμε μόνο ICD-10 με ίδιο αριθμό ≥5 εισαγωγών.

B.15 Βρείτε την κατανομή triage: επίπεδο επείγοντος, μέσος χρόνος αναμονής, ποσοστό εισαγωγής, παραπομπές ανά τμήμα.

Υλοποιούμε το παραπάνω ερώτημα τρέχοντας το παρακάτω query στο phpMyAdmin:

```

1  SELECT
2      t.urgency_level,
3      COUNT(t.triage_id) AS total_cases,
4      ROUND(COUNT(t.triage_id) * 100.0 / (SELECT COUNT(*) FROM triage), 2) AS percentage,
5      ROUND(AVG(TIMESTAMPDIFF(MINUTE, t.arrival_time, t.service_time)), 2) AS wait_time,
6      SUM(CASE WHEN t.outcome = 'ADMITTED' THEN 1 ELSE 0 END) AS admitted_count,
7      ROUND(SUM(CASE WHEN t.outcome = 'ADMITTED' THEN 1 ELSE 0 END) * 100.0 / COUNT(t.triage_id), 2) AS admission_percentage,
8      d.name AS department,
9      COUNT(h.hospitalization_id) AS referrals_per_department
10 FROM triage t
11 LEFT JOIN hospitalization h ON t.hospitalization_id = h.hospitalization_id
12 LEFT JOIN department d ON h.department_id = d.department_id
13 GROUP BY t.urgency_level, d.department_id, d.name
14 ORDER BY t.urgency_level, referrals_per_department DESC;

```

Ξεκινάμε από τον πίνακα triage και κάνουμε LEFT JOIN με hospitalization και department για τις παραπομπές. Χρησιμοποιούμε COUNT(t.triage_id) AS total_cases για τον αριθμό περιστατικών και ROUND(COUNT(t.triage_id) * 100.0 / (SELECT COUNT(*) FROM triage), 2) AS percentage για το ποσοστό κατανομής. Ο μέσος χρόνος αναμονής υπολογίζεται με ROUND(AVG(TIMESTAMPDIFF(MINUTE, t.arrival_time, t.service_time)), 2). Χρησιμοποιούμε SUM(CASE WHEN t.outcome = 'ADMITTED' THEN 1 ELSE 0 END) για τον αριθμό εισαγωγών και το αντίστοιχο ποσοστό. Ομαδοποιούμε ανά urgency_level και department.

Γ. Υλοποίηση Εφαρμογής

Γ.1. Επισκόπηση Εφαρμογής

Η εφαρμογή YGEIOPOLIS είναι χτισμένη με Node.js και Express, με στόχο να παρέχει ένα δυναμικό web-based περιβάλλον για τη διαχείριση των λειτουργιών του νοσοκομείου. Οι χρήστες μπορούν να δουν και να αλληλεπιδράσουν με πληροφορίες για ασθενείς, ιατρικό προσωπικό, τμήματα, νοσηλείες, triage περιστατικά και φαρμακείο, καθώς και να καταχωρούν νέα δεδομένα. Η εφαρμογή είναι δομημένη ώστε να προσφέρει εύκολη και φιλική προς τον χρήστη εμπειρία.

Γ.2. Τεχνολογίες και Εργαλεία

Για την ανάπτυξη της εφαρμογής χρησιμοποιήθηκαν οι εξής τεχνολογίες:

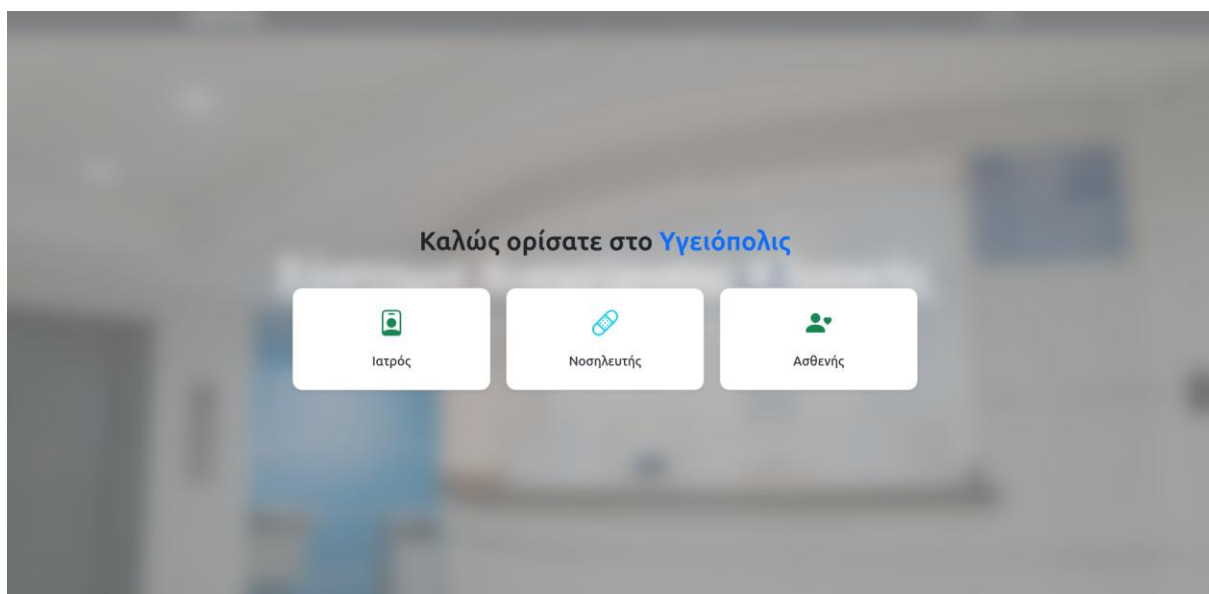
- Node.js: Η πλατφόρμα εκτέλεσης JavaScript για τον server.
- Express: Το web framework για τη δημιουργία του server-side μέρους της εφαρμογής.
- EJS (Embedded JavaScript): Template engine για τη δυναμική δημιουργία HTML σελίδων.
- MySQL2: Connector για τη σύνδεση με τη βάση MariaDB.
- MariaDB 10.4.32: Σύστημα διαχείρισης βάσης δεδομένων.
- HTML/CSS: Γλώσσες για την ανάπτυξη της διεπαφής χρήστη.
- XAMPP: Τοπικό περιβάλλον ανάπτυξης για MariaDB.

Γ.3. Δομή Εφαρμογής

Η εφαρμογή περιλαμβάνει τις παρακάτω βασικές σελίδες:

Αρχική Σελίδα (Role Selection)

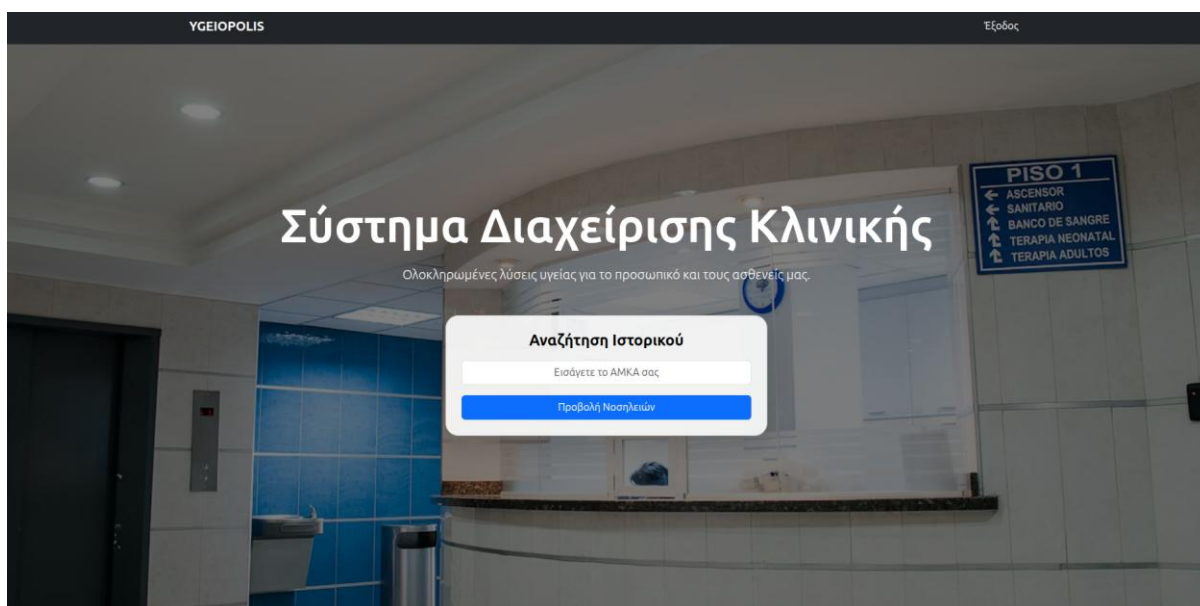
Στην αρχική σελίδα, οι χρήστες μπορούν να επιλέξουν έναν από τρεις ρόλους, Ιατρός, Νοσηλευτής και Ασθενής. Ο κάθε ρόλος μπορεί να διαχειριστεί, να δει διαφορετικά δεδομένα και έχει εν γένει διαφορετικές δυνατότητες στην ιστοσελίδα, οι οποίες παρουσιάζονται παρακάτω. Η συγκεκριμένη υλοποίηση έγινε ως αντικατάστατο σε ένα σύστημα διαχείρισης λογαριασμών, το οποίο θα μπορούσε να περιλαμβάνει Login, Register κλπ.



Εικόνα 1: Σελίδα Επιλογής Ρόλου

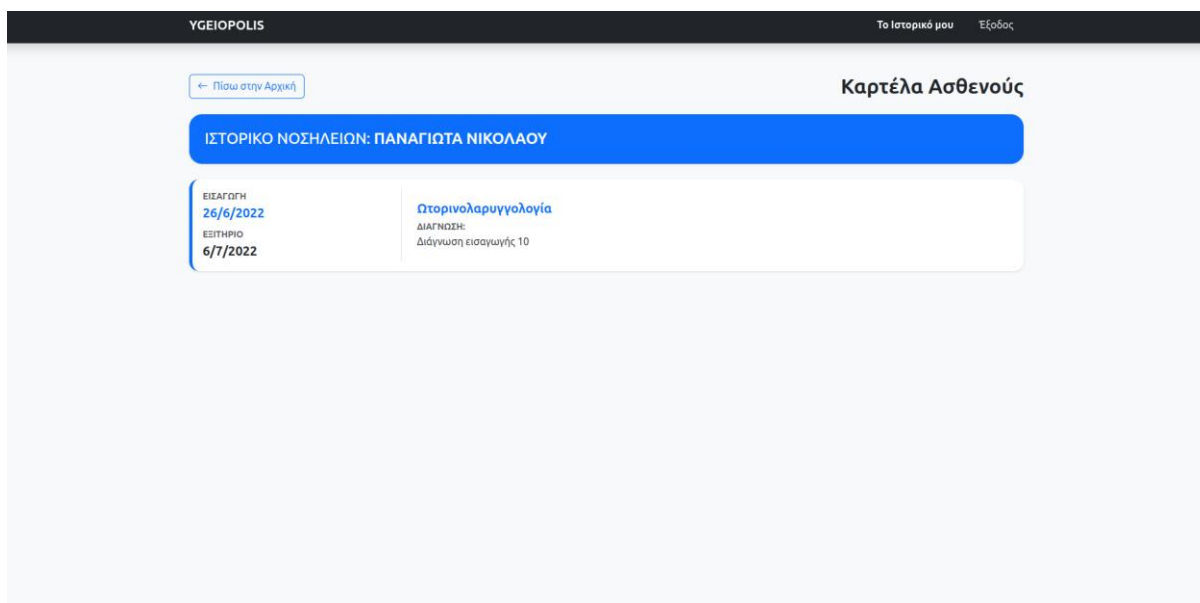
Οθόνη Ασθενών

Η οθόνη ασθενών παρέχει στους ασθενείς τη δυνατότητα αναζήτησης του ιστορικού νοσηλειών τους εισάγοντας τον ΑΜΚΑ τους. Επίσης παρέχει navigation στις υπόλοιπες ενότητες μέσω της navigation bar (Dashboard, Ασθενείς, Triage, Νοσηλείες, Φαρμακείο, Προσωπικό, Τμήματα).



Εικόνα 2: Αρχική Σελίδα Ασθενών

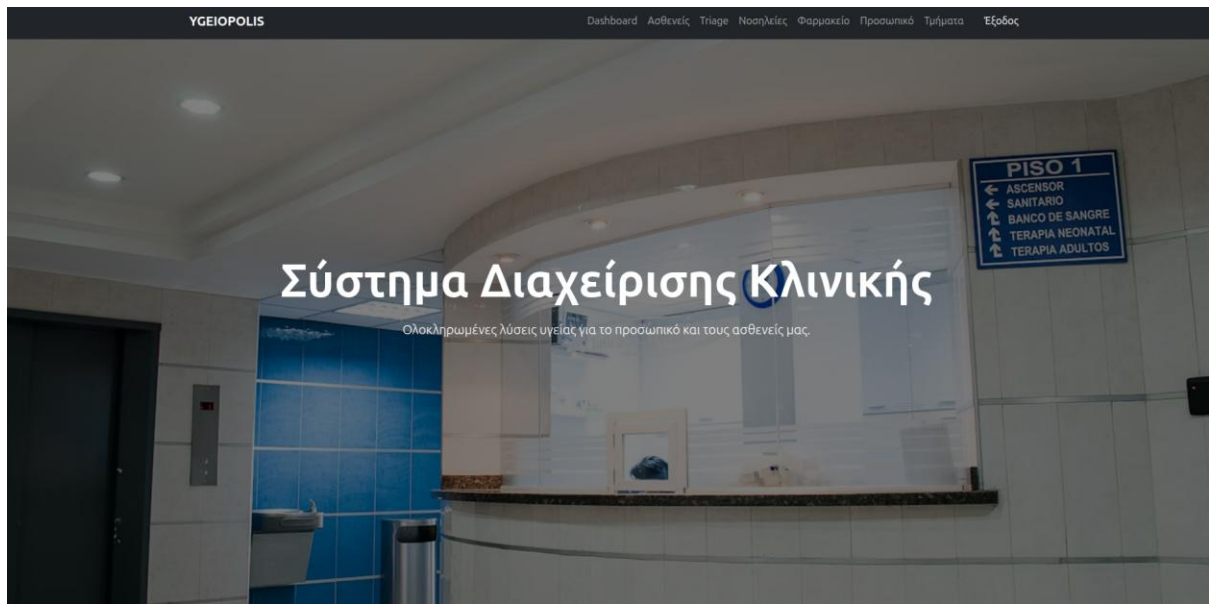
Με την εισαγωγή ενός έγκυρου ΑΜΚΑ, ο ασθενής βλέπει τη καρτέλα του, δηλαδή το ιστορικό νοσηλειών του:



Εικόνα 3: Καρτέλα Ασθενούς – Ιστορικό Νοσηλειών

Οθόνη Ιατρών

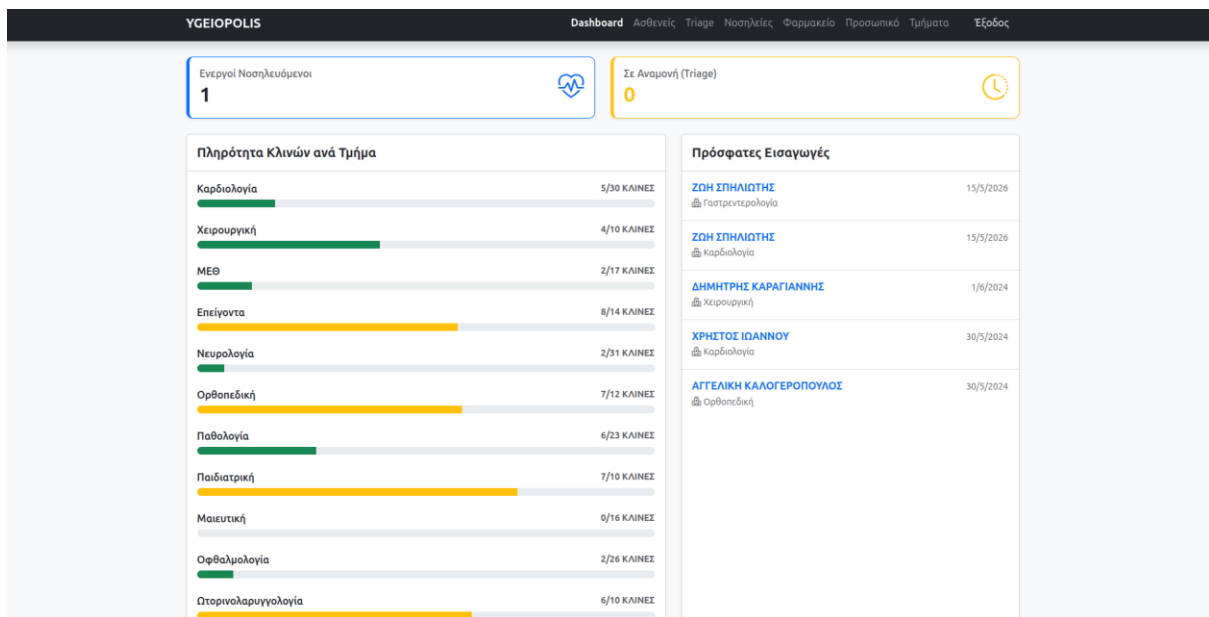
Στον ρόλο των ιατρών παρέχονται όλες οι δυνατότητες της ιστοσελίδας. Αυτές συμπεριλαμβάνουν τις εξής οθόνες, στις οποίες παρέχεται πρόσβαση μέσω του Navigation Bar στο πάνω μέρος της οθόνης. Οι σελίδες στις οποίες έχουν πρόσβαση οι ιατροί είναι: Dashboard, Ασθενείς, Triage, Νοσηλείες, Φαρμακείο, Προσωπικό και Τμήματα. Περισσότερες λεπτομέρειες για την κάθε σελίδα παρέχονται παρακάτω.



Εικόνα 4: Κεντρική Οθόνη Ιατρών

Dashboard

Στο Dashboard, οι ιατροί μπορούν να δουν γενικά στατιστικά στοιχεία του Νοσοκομείου, όπως τους Ενεργούς Νοσηλευόμενους, τον αριθμό Ασθενών που βρίσκονται σε διαδικασία Διαλογής (Triage), τις Πρόσφατες Εισαγωγές και την Πληρότητα Κλινών ανά τμήμα.



Εικόνα 5: Dashboard

Μητρώο Ασθενών

Η σελίδα αυτή εμφανίζει τη λίστα όλων των ασθενών με ΑΜΚΑ, ονοματεπώνυμο, ηλικία και ασφαλιστικό φορέα. Οι ασφαλιστικοί φορείς εμφανίζονται με color-coded badges (π.χ. ΕΦΚΑ, ΥΠΕΔ, NAT, Ιδιωτική Ασφάλεια).

ΥΓΕΙΟΠΟΛΙΣ

Dashboard

Ασθενείς

Τρία

Νοσηλείες

Φαρμακείο

Προσωπικό

Τμήματα

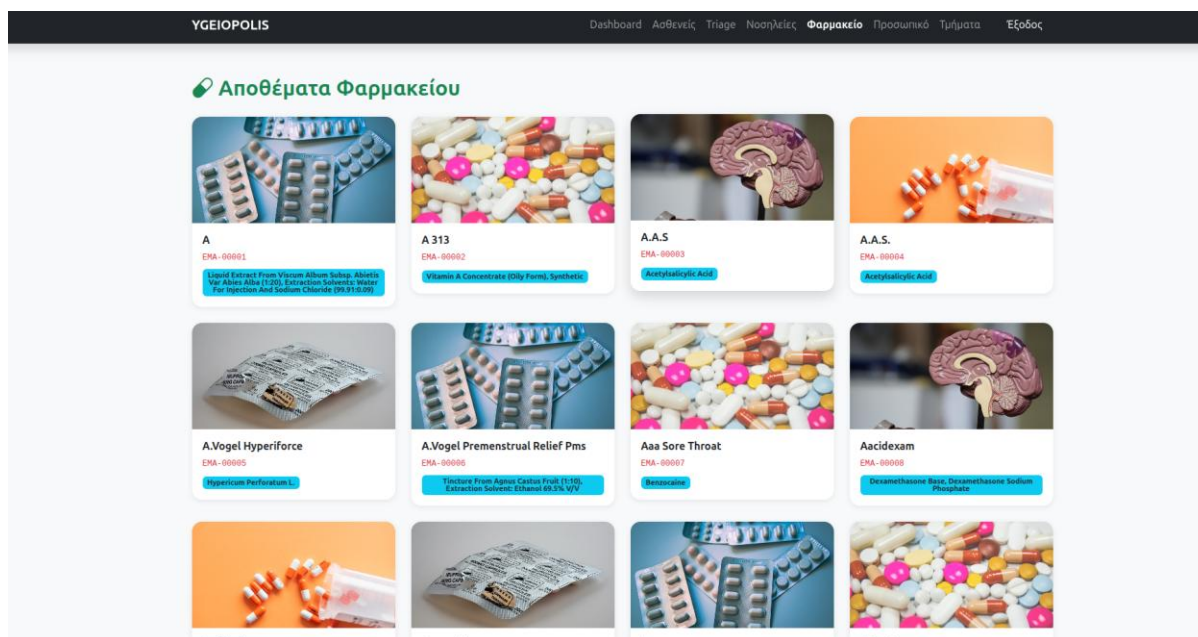
Έξοδος

Μητρώο Ασθενών

ΑΜΚΑ	Όνομα	Επώνυμο	Ηλικία	Ασφάλεια
0000000601	Παναγιώτα	Νικολάου	66	Ιδιωτική Ασφάλεια
0000000602	Χριστίνα	Κωνσταντίνου	74	ΥΠΕΔ
0000000603	Ελένη	Παπαδάκης	64	ΕΦΚΑ
0000000604	Νικολέτα	Παπαδάκης	7	NAT
0000000605	Ειρήνη	Τσακίρης	19	Ιδιωτική Ασφάλεια
0000000606	Ανastasία	Σταύρου	39	Ανοηφάλεια
0000000607	Αγγελική	Αθανασίου	74	ΥΠΕΔ
0000000608	Γιώργης	Καραγιάννης	54	ΕΦΚΑ
0000000609	Φωτεινή	Σταύρου	23	NAT
0000000610	Τάσος	Κοντός	45	NAT
0000000611	Αγγελική	Καλογερόπουλος	28	ΕΦΚΑ
0000000612	Χρήστος	Παπαδάκης	53	NAT
0000000613	Σπύρος	Κοντός	83	ΕΦΚΑ
0000000614	Ανastasία	Καλογερόπουλος	5	NAT
0000000615	Αγγελική	Σταματίου	2	NAT
0000000616	Τάσος	Κοντός	1	NAT
0000000617	Δημήτρης	Καραγιάννης	21	Ανοηφάλεια

Ιατρικό Προσωπικό

Η σελίδα αυτή εμφανίζει το ιατρικό προσωπικό με ονοματεπώνυμο, ειδικότητα, βαθμίδα και αριθμό ιατρικής άδειας. Οι ειδικότητες εμφανίζονται με color-coded badges για εύκολη αναγνώριση.



Εικόνα 8: Αποθέματα Φαρμακείου

Triage – Καταγραφή Περιστατικού

Η σελίδα Triage εμφανίζει τα ενεργά περιστατικά αναμονής ταξινομημένα κατά επίπεδο επείγοντος και ώρα άφιξης. Παρέχεται φόρμα καταγραφής νέου περιστατικού με ΑΜΚΑ ασθενή, SSN νοσηλευτή, επίπεδο επείγοντος (1-Άμεσο έως 5-Μη Επείγον) και συμπτώματα. Επίσης παρέχεται η δυνατότητα άμεσης εισαγωγής σε νοσηλεία.

Νέα Καταγραφή Περιστατικού
✕

SSN Ασθενούς (ΑΜΚΑ)

00000000601

SSN Νοσηλευτή Διαλογής

00000000201

Επίπεδο Επείγοντος

1 - Άμεσο (Κόκκινο) ▼

Συμπτώματα

Άκυρο

Καταχώρηση

Εικόνα 9: Φόρμα Νέας Καταγραφής Triage

Σε περίπτωση που το ΑΜΚΑ του Ασθενούς δεν υπάρχει μέσα στη βάση, τότε ανοίγει μια νέα φόρμα για την εισαγωγή νέου Ασθενούς:

Εικόνα 10: Φόρμα Εγγραφής νέου Ασθενούς

Μετά την Καταχώρηση Διαλογής, οι Ιατροί έχουν τη δυνατότητα να Εισάγουν τον Ασθενή σε Νοσηλεία.

Αφίξη	Ασθενής	Επίπεδο	Συμπτώματα	Ενέργειες
1:19:03 μ.μ.	Ζωή Σπηλιώτης	L1		Εισαγωγή

Εικόνα 11: Οθόνη Διαλογών

YGEIOPOLIS

DashboardΑσθενείςTriageΝοσηλείεςΦαρμακείοΠροσωπικάΤμήματαΕξοδος

Φόρμα Εισαγωγής Ασθενούς

Ασθενής: Ζωή Σπηλιώτης (SSN: 00000000729)

Συμπτώματα:

Επιλογή Διαθέσιμης Κλίνης

Γαστρεντερολογία - Κρεβάτι: K14-001

Κωδικός KEN (DRG)

G01Ma - Χειρουργικές επεμβάσεις που δεν σχετίζονται με κάποια κύρια διάγνωση με καταστροφικές συνυ

Κωδικός ICD10

A00

Περιγραφή Διάγνωσης

Πληκτρολογήστε τη διάγνωση...

Ακύρωση

Ολοκλήρωση Εισαγωγής

Εικόνα 12: Φόρμα Εισαγωγής Ασθενούς

Νοσηλείες

Στην συγκεκριμένη οθόνη μπορούν οι ιατροί να δουν τις Ενεργές Νοσηλείες, και παρέχονται ορισμένες ενέργειες σε αυτούς, η Συνταγογράφηση, Εξετάσεις και η Έκδοση Εξιτηρίου

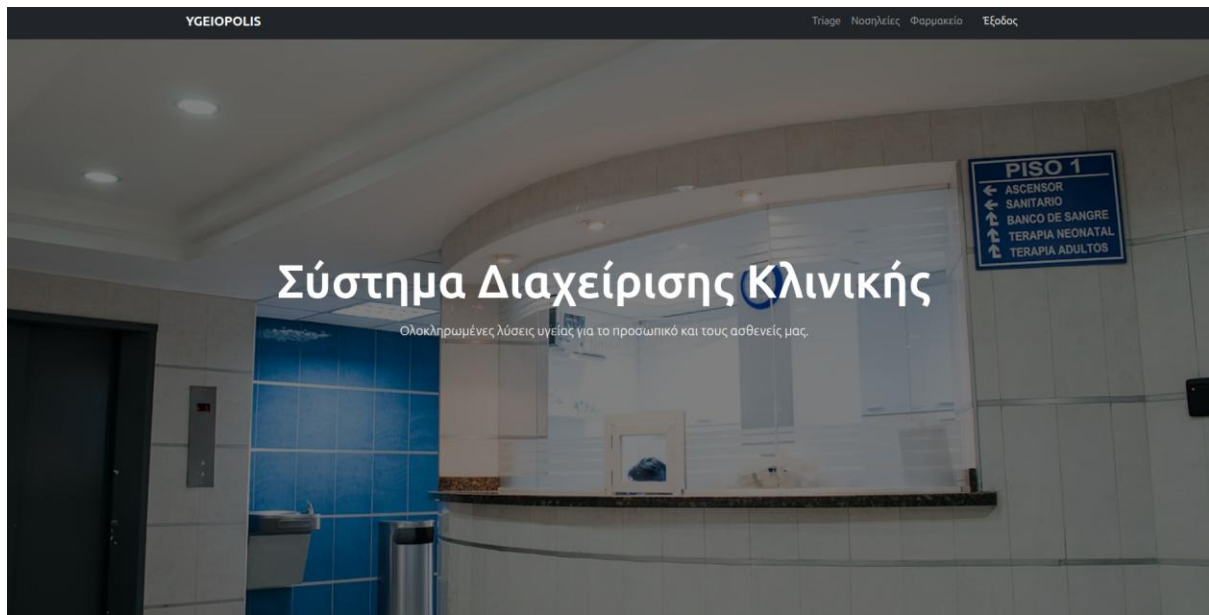
YGEIOPOLIS

DashboardΑσθενείςTriageΝοσηλείεςΦαρμακείοΠροσωπικάΤμήματαΕξοδος

Ενεργές Νοσηλείες

ID	Ασθενής	Τμήμα	Κρεβάτι	Εισαγωγή	Ενέργειες
#607	Ζωή Σπηλιώτης	Γαστρεντερολογία	No. K14-001	19/5/2026	  
#606	Ζωή Σπηλιώτης	Καρδιολογία	No. K01-007	15/5/2026	  

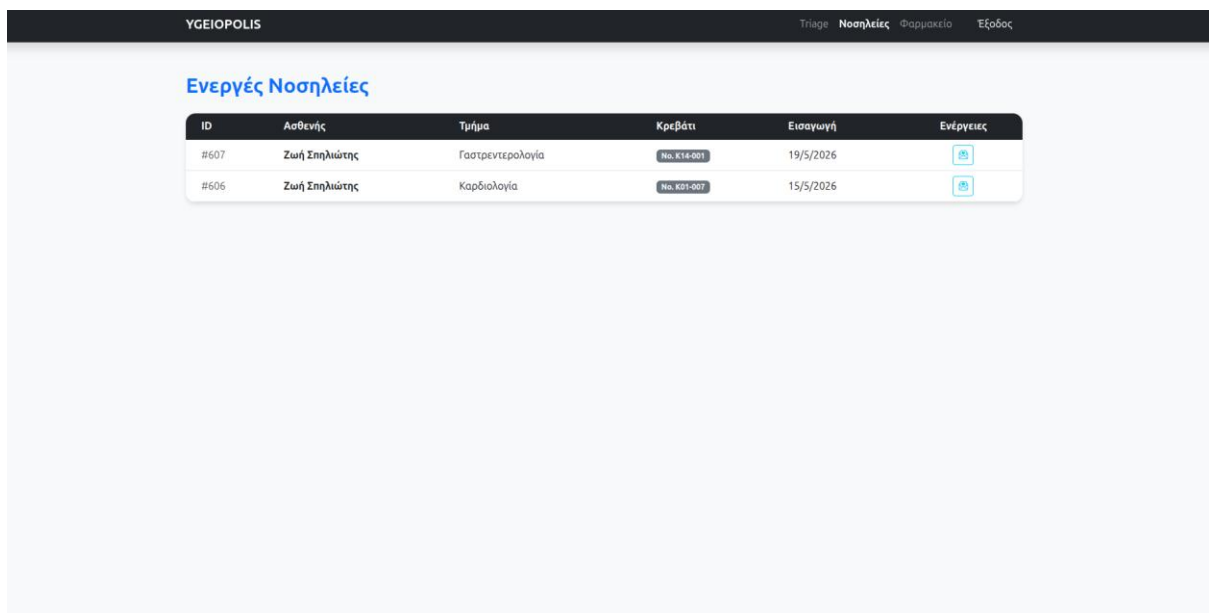
Εικόνα 13: Ενεργές Νοσηλείες



Εικόνα 16: Κεντρική Οθόνη Νοσηλευτών

Στην οθόνη Διαλογής μπορούν να καταχωρήσουν μια νέα Διαλογή και έπειτα να εισάγουν τον Ασθενή σε Νοσηλεία, όπως ακριβώς οι Ιατροί.

Στην σελίδα των Νοσηλειών, βλέπουν τη λίστα των Ενεργών Νοσηλειών, με τη μόνη διαφορά ότι δεν μπορούν να Συνταγογραφήσουν και να εκδώσουν εξιτήριο. Σε σύγκριση δηλαδή με τον ρόλο των Ιατρών, έχουν μόνο πρόσβαση στις Εξετάσεις του Ασθενούς.



Εικόνα 17: Οθόνη Νοσηλειών για τους Νοσηλευτές

Γ.4. Οδηγίες Εγκατάστασης και Χρήσης

Για την εκτέλεση της εφαρμογής πρέπει να ακολουθηθούν τα εξής βήματα:

1. Εγκατάσταση Node.js (έκδοση 18 ή νεότερη) από <https://nodejs.org>

2. Εγκατάσταση XAMPP και εκκίνηση Apache και MySQL από το XAMPP Control Panel.
3. Δημιουργία βάσης δεδομένων ygeiopolis μέσω phpMyAdmin.
4. Import του αρχείου sql/install.sql για τη δημιουργία του σχήματος (πίνακες, indexes, triggers).
5. Import του αρχείου sql/load.sql για τη φόρτωση των δεδομένων.
6. Μεταβείτε στον φάκελο Code: `cd C:/xampp/htdocs/Databases-Project-2026/Code`
7. Εκτέλεση: `npm install` (εγκατάσταση των dependencies)
8. Εκτέλεση: `node app.js`
9. Άνοιγμα browser στη διεύθυνση: `http://localhost:3000`

Δήλωση Χρήσης Εργαλείων ΤΝ

Σύμφωνα με τις οδηγίες ακαδημαϊκής δεοντολογίας του ΕΜΠ/ΣΗΜΜΥ, δηλώνουμε ότι εργαλεία Τεχνητής Νοημοσύνης χρησιμοποιήθηκαν για τη δημιουργία των dummy data (load.sql) και για βοήθεια στην ανάπτυξη του front-end (HTML/CSS). Όλες οι σχεδιαστικές αποφάσεις, η υλοποίηση της βάσης δεδομένων, τα SQL ερωτήματα, οι triggers και οι indexes υλοποιήθηκαν από τα μέλη της ομάδας.